Adhyan — System Architecture

Adaptive, gamified quizzing platform for college students. Last updated: 2026-06-03 · Status: working MVP (demo mode) + production scaffolding

This document describes the complete system: tech stack, layered architecture, data model, the four core engines (transformation, grading, progression, placement), request/data flows, security model, and the path to production.

---

1. Executive Overview

Adhyan lets **teachers** publish paid courses and **students** climb through them level by level. Its defining properties:

- **Anti-theft by design** — question answer keys never reach the browser; grading is server-side; every student sees a unique generated variant of each question.
- **Adaptive entry** — a 10-question placement test decides which level a student starts at.
- **Difficulty-only levels** — within a topic, levels are pure difficulty steps (Level 1 → N); concepts are never surfaced.
- **Gamified UX** — XP, streaks, badges, skill-tree map, timed quizzes, celebratory feedback.
- **Per-course subscriptions** via Razorpay.

The codebase is structured so the **domain logic is pure and backend-agnostic**: it runs today on an in-memory demo store and will run unchanged on Supabase by swapping a single data layer.

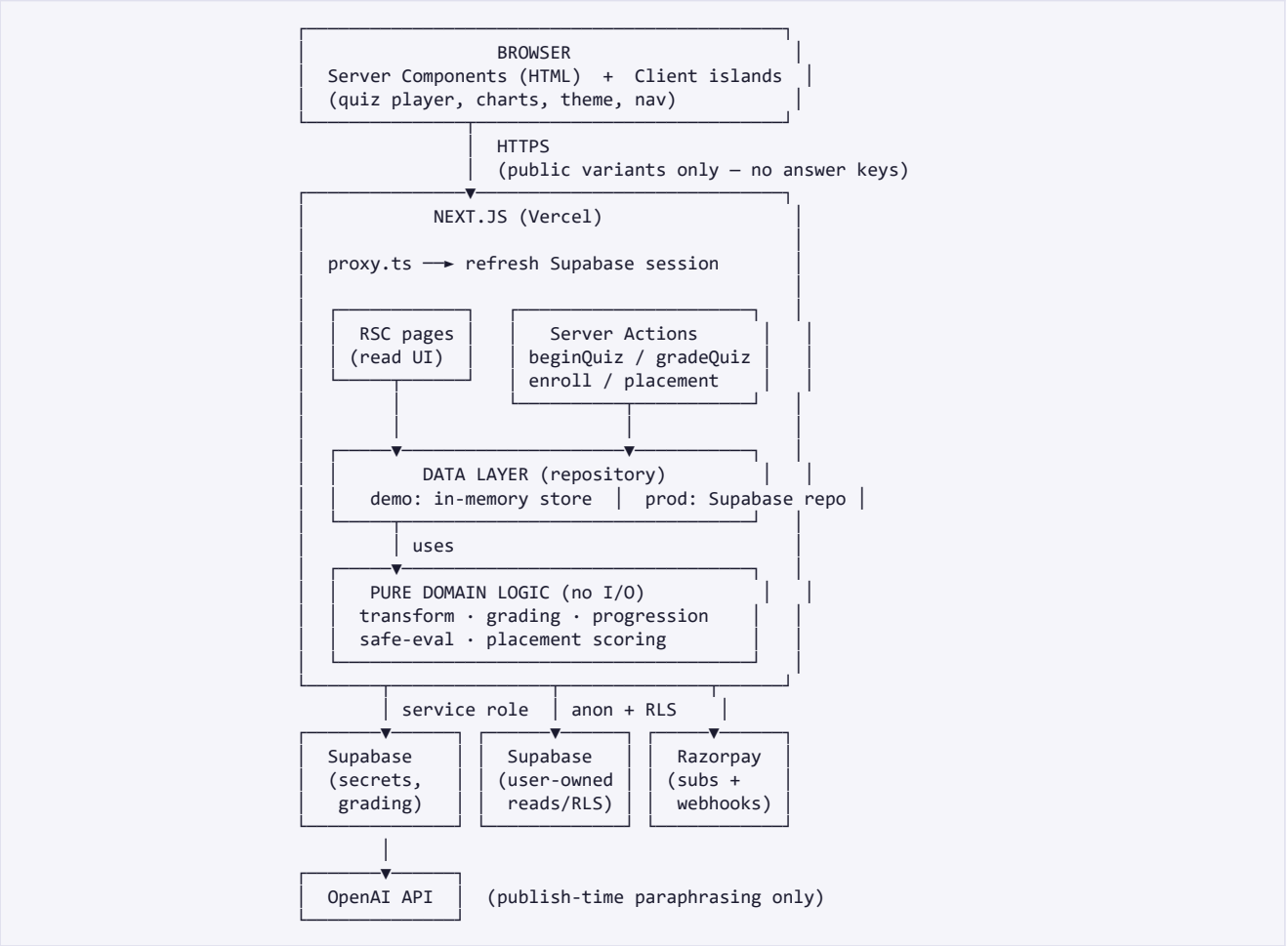
---

2. Technology Stack

Layer	Technology	Notes
Framework	**Next.js 16** (App Router, Turbopack, RSC + Server Actions)	`proxy.ts` (formerly middleware) for session refresh
Language	**TypeScript 5** (strict)	Shared types across domain, data, UI
UI runtime	**React 19**	Server Components by default; Client Components only where interactive
Styling	**Tailwind CSS v4** (CSS-first `@theme`)	Custom design tokens, light/dark via `.dark` class
Components	**shadcn-style primitives** + Radix Slot	`Button`, `Card`, `Badge`, `Progress`, `Ring` (hand-rolled, lightweight)
Icons	**lucide-react**	
Animation	**framer-motion**	Quiz transitions, result celebration, confetti
Charts	**recharts**	Weekly XP activity (client-only, mount-guarded)
Theming	**next-themes**	Persistent light/dark
Validation	**zod**	(available for input/schema validation)
Backend / DB / Auth	**Supabase** (Postgres + Auth + Storage + RLS + Realtime)	via `@supabase/ssr` and `@supabase/supabase-js`
Payments	**Razorpay**	Per-course subscriptions (webhook-driven)
AI	**OpenAI API**	Theory-question paraphrasing only — never computes answers
Hosting	**Vercel** (Fluid Compute)	Static + dynamic routes, Server Actions, Cron

---

3. High-Level Architecture



Key principle: two distinct paths into Supabase —

1. **Client + anon key + RLS** for user-owned reads (profile, progress, enrollments, leaderboards).
2. **Server + service role** (bypasses RLS) for anything touching answer keys or progression: serving questions, grading, recomputing unlocks, payment webhooks.

---

4. Layered Architecture

The app is split into four layers with strict dependency direction (UI → actions → data → domain). The domain layer depends on nothing.



Why this matters: the **UI and domain never change** when we move from demo to Supabase — only `store.ts` is replaced by a Supabase repository implementing the same function signatures.

---

5. Project Structure

```
Adhyan Project/
├─ src/
│  ├─ app/
│  │  ├─ layout.tsx                # root: fonts, ThemeProvider, metadata
│  │  ├─ globals.css               # Tailwind v4 theme tokens, light/dark
│  │  ├─ page.tsx                  # landing (marketing)
│  │  ├─ actions.ts                # all Server Actions ("use server")
│  │  ├─ login/page.tsx            # auth (Supabase) + role toggle
│  │  ├─ teacher/page.tsx          # Teacher Studio (curriculum + live variant preview)
│  │  ├─ quiz/[attemptId]/page.tsx  # quiz/placement player host (dynamic)
│  │  └─ (app)/                    # authed student shell (route group)
│  │     ├─ layout.tsx             # top nav + bottom tab bar (XP/streak)
│  │     ├─ dashboard/page.tsx     # home: continue/onboard, stats, mastery, badges
│  │     ├─ learn/page.tsx         # COURSE CATALOG
│  │     ├─ learn/[courseId]/page.tsx  # course → topics → Level N skill tree
│  │     └─ progress/page.tsx      # rings, weekly chart, weak-area, badges
│  ├─ components/
│  │  ├─ app-nav.tsx               # client: nav + active state
│  │  ├─ theme-provider.tsx, theme-toggle.tsx
│  │  ├─ progress-chart.tsx        # client: recharts area chart
│  │  ├─ ui/                       # button, card, badge, progress, ring
│  │  ├─ quiz/
│  │  │  ├─ quiz-player.tsx        # client: timer, one-Q-at-a-time, submit
│  │  │  ├─ result-screen.tsx      # client: score, XP, unlocks, review
│  │  │  └─ confetti.tsx           # client: celebratory animation
│  │  └─ teacher/variant-preview.tsx# client: "what 3 students see"
│  ├─ lib/
│  │  ├─ utils.ts                  # cn(), clamp(), formatClock()
│  │  ├─ domain/                   # PURE LOGIC
│  │  │  ├─ types.ts               # all data shapes
│  │  │  ├─ safe-eval.ts           # arithmetic evaluator (owns math answers)
│  │  │  ├─ transform.ts           # variant generation engine
│  │  │  ├─ grading.ts             # server-side grading
│  │  │  └─ progression.ts         # level/topic unlock + mastery + XP
│  │  ├─ data/
│  │  │  ├─ seed.ts                # demo courses + question bank
│  │  │  └─ store.ts               # in-memory repository (server-only)
│  │  ├─ supabase/
│  │  │  ├─ client.ts              # browser client (anon + RLS)
│  │  │  └─ server.ts              # server client (RLS) + service-role client
│  │  └─ ai/paraphrase.ts          # OpenAI theory paraphrasing (graceful no-op)
│  └─ proxy.ts                     # Next 16 proxy: Supabase session refresh
├─ supabase/migrations/
│  ├─ 0001_init.sql                # schema (with question_secrets split)
│  └─ 0002_rls.sql                 # Row Level Security policies + signup trigger
├─ .env.example                    # Supabase / OpenAI / Razorpay keys
├─ next.config.ts                  # turbopack.root pin
├─ PROJECT_PLAN.md                 # product plan & decisions
├─ CLARIFY_WITH_PROFESSOR.md       # open assumptions to confirm
└─ ARCHITECTURE.md                 # (this file)
```

---

6. Domain Model

Content hierarchy: **Course → Topic → Level → Question**. A level is a pure difficulty step; questions inside one level share a difficulty.

```
Course (subscription, price, trial)
  └─ Topic (ordered; unlockLevelIndex = gate to next topic)
       └─ Level ("Level N", passThreshold y%, minQuestions, timeLimitSec)
            └─ Question (prompt, type, variables, allowedTransforms, …)
                 └─ [SECRET] answer key  ← lives apart; server-only
```

Core types (`src/lib/domain/types.ts`):

Type	Purpose
`AuthoredQuestion`	Teacher-authored template: prompt with `{vars}`, `variables`, `answerFormula`, `options`, `allowedTransforms`, `difficulty`.
`QuestionVariant`	A generated per-student rendering **with** `answerKey` (server-only).

Type	Purpose
`PublicVariant`	`QuestionVariant` minus the answer key — the only thing sent to the browser.
`Level` / `Topic` / `Course`	Content structure; `concept` on a level is an author-only note, never shown.
`LevelProgress`	Per (student, level): status (`locked`/`unlocked`/`in_progress`/`passed`), bestCorrect, percent.
`Gamification`	XP, streakDays, badges.
`SubmitOutcome` / `PlacementOutcome`	Result of grading a level / placement attempt.

---

7. The Four Core Engines

7.1 Transformation Engine (`transform.ts` + `safe-eval.ts`)

Turns one `AuthoredQuestion` into a **deterministic per-student variant** so no two students see the same question, and a screenshot is low-value.

- **Seeded RNG** (mulberry32) keyed by `hashSeed(attemptId, questionId)` → same student+attempt always regenerates the identical variant (refresh-safe, cacheable).
- **Path A — Parameterized templates:** randomize `{vars}` within constraints; compute the answer via `safeEval(answerFormula, vars)`.
- **Path B — Format transformation:** render the same question as single-choice / multiple / integer / fill-in-blank / true-false (drawn from `allowedTransforms`).
- **Path C — Surface variation:** shuffle options; generate plausible numeric distractors around the correct value.
- Output: a full `QuestionVariant` (server) and, via `toPublicVariant()`, a `PublicVariant` (client).

> **Correctness boundary:** `safe-eval.ts` is a hand-written arithmetic evaluator (`+ - * / % ^`, parentheses, variables) — **not** `eval`. It *owns* quantitative answers. The AI is **never** allowed to compute an answer (it would hallucinate); OpenAI is used only for wording/distractors of theory questions, at publish time, with teacher approval.

7.2 Grading (`grading.ts`)

- Pure function `gradeAnswer(variant, submittedAnswer) → boolean`, run **on the server only**.
- Handles choice answers (set-equality of option ids) and value answers (numeric with **2-decimal rounding tolerance**, or normalized text).
- The browser submits a raw answer; it never possesses the key.

7.3 Progression (`progression.ts`)

- **Level pass:** `bestCorrect ≥ ceil(passThreshold% × totalQuestions)` (best-attempt-per-question scoring; unlimited retries).
- **Sequential unlock:** within a topic, level *n+1* unlocks when level *n* is passed.
- **Topic gate:** a topic unlocks when the previous topic's `unlockLevelIndex` level is passed.
- `computeCourseStatuses()` recomputes every level's status from the progress map (called after each attempt).
- Also provides `topicMastery`, `courseCompletion`, `nextActionableLevel`, `xpForAttempt`.

7.4 Placement (in `store.ts`, using domain logic)

- After enrolling, a student takes a **10-question diagnostic** spanning all difficulty levels (round-robin across the course's levels).
- Score → place-out mapping: `0–3→0, 4–5→1, 6–7→2, 8–9→3, 10→4` levels skipped (capped so the last level remains).
- Placed-out levels are marked `passed`; the student starts at the computed level (returned as *Topic · Level N*). Timer: 10 minutes.

---

8. Key Data Flows

8.1 Take a timed quiz (anti-theft critical path)

```
Student clicks "Start" (a <form action={beginQuiz}>)
   │
   ▼  Server Action: beginQuiz(levelId)
store.startAttempt(levelId)
   • gate-check level is unlocked
   • generate variants (answer keys) → store SERVER-SIDE under attemptId
   • return PUBLIC variants (no keys)
   │  redirect → /quiz/[attemptId]
   ▼
RSC /quiz/[attemptId] → getAttemptView() → <QuizPlayer> (client)
   • server-authoritative countdown (deadline stored server-side)
   • one question at a time; collects answers locally
   │
   ▼  Server Action: gradeQuiz(attemptId, answers)
store.submitAttempt()
   • grade each answer SERVER-SIDE against stored keys
   • applyAttempt → update progress, detect unlocks, award XP/badges
   • cache outcome (idempotent — see note)
   • return SubmitOutcome (your answers + correct answers + explanations)
   ▼
<ResultScreen>: score ring, XP, "new content unlocked", confetti, review
```

> **Idempotency note:** a Server Action auto-revalidates the current route, which re-runs the quiz page. To avoid a 404 (attempt consumed) and double-XP, `submitAttempt` marks the attempt submitted and **returns a cached outcome** on re-entry instead of deleting it.

8.2 Enroll → placement → start at the right level

```
Catalog/course page → <form action={enrollCourse(courseId)}>
   → enroll() (creates enrollment / trial)
   → startPlacement() → redirect /quiz/[attemptId]   (10 mixed-level Qs)
        → gradeQuiz → finishPlacement():
             • score → placedCount
             • mark placed-out levels "passed"
             • return PlacementOutcome (Topic · Level N start)
   → ResultScreen ("You're all set! Starting you at <Topic> · Level N")
   → course page shows "Placed out" levels + the start level unlocked
```

8.3 Subscription payment (production design)

```
Enroll → create Razorpay subscription (per-course Plan)
   → Razorpay webhook (server, service role) is the source of truth:
        charged → enrollment.active ; halted/cancelled → re-lock content
   → daily Cron reconciles enrollment status as a backstop
```

8.4 Teacher variant preview

```
Teacher Studio → "Preview variants" → Server Action previewQuestionVariants()
   → generate 3 fresh variants of one authored question (teacher MAY see answers)
   → renders "what 3 different students would see" + the correct answer
```

9. Rendering & Performance Strategy

- **Server Components by default**; client JS limited to true interactivity (quiz player, charts, theme, nav active-state, variant preview).
- **Static prerender** for `/`, `/dashboard`, `/learn`, `/progress`, `/teacher`, `/login`; **dynamic** for `/quiz/[attemptId]` and `/learn/[courseId]`.
- Animations use **GPU-friendly transforms** (framer-motion / CSS) — confetti is pure transform, no layout thrash.
- Recharts is **mount-guarded** to skip SSR (no zero-size warning, no hydration mismatch).
- Mobile-first, responsive; bottom tab bar on small screens.

10. Security Model

Concern	Mitigation
Answer-key theft	Keys live in `question_secrets` (service-role only) and in server-held variants; never serialized to the client. Grading is server-side.
Bulk scraping	Questions are drip-fed one attempt at a time; no "fetch all questions" endpoint exists.
Per-student leakage	Deterministic per-student variants make a leaked screenshot rarely match anyone; forensic watermarking planned.
Authorization	RLS scopes user-owned rows (students read only their own progress/attempts/enrollments). Teachers write only their own courses.
Timer tampering	Deadlines are server-authoritative; late submissions rejected; refresh returns server-computed remaining time.
Payment spoofing	Access granted only via verified Razorpay **webhooks**, never client redirects.
AI hallucination	AI never computes/decides answers; deterministic evaluator owns them; AI output is teacher-approved.
Session integrity	`proxy.ts` refreshes Supabase sessions per request.

11. Database (Supabase) — Production

Defined in `supabase/migrations/`:

- `0001_init.sql` — tables: `profiles`, `courses`, `topics`, `levels`, `questions`, `question_secrets` (answer keys, split out), `enrollments` (Razorpay subscription fields), `attempts`, `answer_records`, `progress`, `gamification`, `xp_history`. Enums for roles, question types, level/enrollment status. Indexes on hot foreign keys.
- `0002_rls.sql` — RLS enabled on every table:
  - Public can read **published** course/topic/level metadata.
  - Students read only their **own** enrollments/attempts/progress/gamification.
  - `questions` / `question_secrets` **have no client read policy** — served exclusively via service role.
  - `handle_new_user()` trigger bootstraps `profiles` + `gamification` on signup.

The service role (server only) bypasses RLS for grading, question serving, progression writes, and webhooks.

12. Demo Mode vs Production

Aspect	Demo (today)	Production (next)
Data store	`src/lib/data/store.ts` (in-memory, `globalThis`)	Supabase repository with identical API
Content	`seed.ts` (2 courses, full question bank)	`courses`/`questions`/`question_secrets` tables
Auth	optional; app explorable without login	Supabase Auth (email/password, roles) enforced
Payments	simulated enrollment	Razorpay subscriptions + webhooks
AI paraphrase	no-op without key	OpenAI at publish time, teacher-approved
Persistence	resets on server restart	durable Postgres

Migration effort is intentionally small: replace `store.ts` with a Supabase-backed module exposing the same functions (`getCourses`, `startAttempt`, `submitAttempt`, `enroll`, `startPlacement`, …). UI, Server Actions, and all domain logic stay untouched.

---

13. Deployment (Vercel)

- **Build:** `next build` (Turbopack). Clean build, zero warnings.
- **Runtime:** Fluid Compute; Server Actions and dynamic routes run as functions; static pages prerendered/cached.
- **Env vars** (`.env.example`): `NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_ANON_KEY`, `SUPABASE_SERVICE_ROLE_KEY`, `OPENAI_API_KEY`, `RAZORPAY_KEY_ID/SECRET/WEBHOOK_SECRET`.
- **Cron** (planned): daily enrollment reconciliation; streak resets; nightly variant pre-generation.
- **Local dev:** `npm run dev` (port 3000).

---

14. Roadmap to Production

1. Provision Supabase; run the two migrations; seed real content.
2. Implement the Supabase repository (same interface as `store.ts`); flip the data layer.
3. Enforce Supabase Auth + roles; gate student/teacher routes.
4. Razorpay subscription checkout + webhook handler + reconciliation Cron.
5. Enable OpenAI paraphrasing at publish time (teacher approval UI).
6. Add forensic watermarking, rate-limit/anti-scrape guards.
7. Teacher authoring (create/edit, CSV import, drag-drop curriculum).
8. PWA, push notifications, spaced repetition, leaderboards (Supabase Realtime).

---

15. Cross-References

- `PROJECT_PLAN.md` — product vision, decisions, pricing, feature backlog.
- `CLARIFY_WITH_PROFESSOR.md` — open assumptions to confirm.
- `supabase/migrations/` — authoritative schema + RLS.
- `src/lib/domain/` — the pure, testable heart of the system.